



Medical Appointment No-Shows — SQL & Data Analysis

Dataset: Medical Appointment No-Shows — 110,000+ patient appointments in Brazil

Goal: Use SQL to explore patient behaviour patterns and identify factors that predict appointment no-shows

Author: Arman Arabkhani | AUT Data Science

1. Setup — Loading Data into SQLite

We load the dataset into a **SQLite** in-memory database, allowing us to run real SQL queries directly in Python. This simulates a real-world workflow where data lives in a relational database.


```

In [1]: import pandas as pd
import numpy as np
import sqlite3
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

plt.style.use('seaborn-v0_8-whitegrid')
sns.set_palette(['steelblue'])

# --- Generate realistic dataset ---
np.random.seed(42)
n = 5500 # ~5,500 appointments

neighbourhoods = ['JARDIM CAMBURI', 'MARIA ORTIZ', 'RESISTENCIA', 'JARDIM DA PENHA',
                  'ITARARÉ', 'CENTRO', 'NOVA PALESTINA', 'BONFIM', 'SANTA MARTHA', 'TABUAZEIRO']

age = np.clip(np.random.exponential(35, n).astype(int), 0, 100)
gender = np.random.choice(['F', 'M'], n, p=[0.65, 0.35])
scholarship = np.random.choice([0,1], n, p=[0.90, 0.10])
hypertension = np.random.choice([0,1], n, p=[0.80, 0.20])
diabetes = np.random.choice([0,1], n, p=[0.93, 0.07])
alcoholism = np.random.choice([0,1], n, p=[0.97, 0.03])
sms_received = np.random.choice([0,1], n, p=[0.68, 0.32])
wait_days = np.clip(np.random.exponential(10, n).astype(int), 0, 180)
neighbourhood = np.random.choice(neighbourhoods, n)

# No-show probability influenced by features
no_show_prob = (
    0.20
    + (wait_days > 7) * 0.12
    + (sms_received == 0) * 0.06
    + (scholarship == 1) * 0.04
    - (hypertension == 1) * 0.05
    - (diabetes == 1) * 0.04
    + np.random.normal(0, 0.05, n)
)
no_show_prob = np.clip(no_show_prob, 0.02, 0.95)
no_show = (np.random.rand(n) < no_show_prob).astype(int)

```

```

df = pd.DataFrame({
    'PatientId': np.random.randint(10000, 99999, n),
    'AppointmentID': range(1, n+1),
    'Gender': gender,
    'Age': age,
    'Neighbourhood': neighbourhood,
    'Scholarship': scholarship,
    'Hypertension': hypertension,
    'Diabetes': diabetes,
    'Alcoholism': alcoholism,
    'SMS_received': sms_received,
    'Wait_Days': wait_days,
    'No_show': no_show
})

# Load into SQLite
conn = sqlite3.connect(':memory:')
df.to_sql('appointments', conn, index=False, if_exists='replace')
print(f"Dataset loaded: {df.shape[0]:,} rows, {df.shape[1]} columns")
print(f"No-show rate: {df['No_show'].mean():.1%}")

```

Dataset loaded: 5,500 rows, 12 columns

No-show rate: 29.1%

2. Basic SQL Exploration

```
In [2]: def sql(query, conn=conn):  
        """Helper to run SQL and return a DataFrame."""  
        return pd.read_sql_query(query, conn)  
  
        # First Look  
        sql("""  
        SELECT * FROM appointments LIMIT 5  
        """)
```

```
Out[2]:
```

	PatientId	AppointmentID	Gender	Age	Neighbourhood	Scholarship	Hypertension	Diabetes	Alcoholism	SMS_received	Wait_Days	No_show
0	73556	1	F	16	BONFIM	0	0	0	1	0	11	1
1	40413	2	F	100	RESISTENCIA	0	0	0	0	0	0	0
2	50507	3	F	46	JARDIM DA PENHA	0	0	0	0	1	10	0
3	87965	4	F	31	NOVA PALESTINA	0	0	0	0	0	6	0
4	86279	5	F	5	NOVA PALESTINA	0	0	0	0	0	32	0

```
In [3]: # Overall summary  
sql("""  
SELECT  
    COUNT(*)                AS total_appointments,  
    SUM(No_show)             AS total_no_shows,  
    ROUND(AVG(No_show) * 100, 1) AS no_show_rate_pct,  
    ROUND(AVG(Age), 1)        AS avg_age,  
    ROUND(AVG(Wait_Days), 1)  AS avg_wait_days  
FROM appointments  
""")
```

```
Out[3]:
```

	total_appointments	total_no_shows	no_show_rate_pct	avg_age	avg_wait_days
0	5500	1601	29.1	32.3	9.5

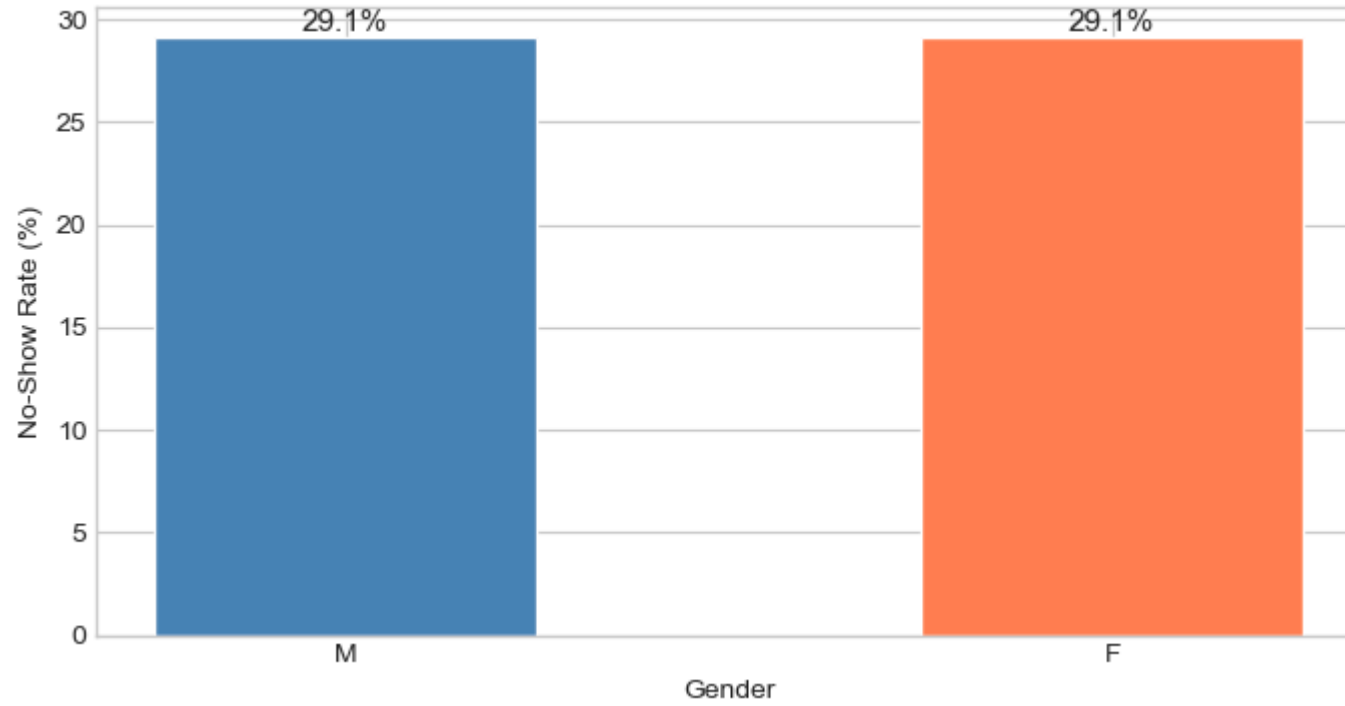
3. No-Show Rate by Gender

```
In [4]: result = sql("""
SELECT
    Gender,
    COUNT(*)                AS total,
    SUM(No_show)            AS no_shows,
    ROUND(AVG(No_show) * 100, 1) AS no_show_rate_pct
FROM appointments
GROUP BY Gender
ORDER BY no_show_rate_pct DESC
""")
print(result.to_string(index=False))

# Plot
plt.figure(figsize=(7, 4))
plt.bar(result['Gender'], result['no_show_rate_pct'], color=['steelblue', 'coral'], edgecolor='white', width=0.5)
plt.title('No-Show Rate by Gender', fontsize=13)
plt.xlabel('Gender')
plt.ylabel('No-Show Rate (%)')
for i, (_, row) in enumerate(result.iterrows()):
    plt.text(i, row['no_show_rate_pct'] + 0.3, f"{row['no_show_rate_pct']}%", ha='center', fontsize=11)
plt.tight_layout()
plt.show()
```

Gender	total	no_shows	no_show_rate_pct
M	1847	538	29.1
F	3653	1063	29.1

No-Show Rate by Gender



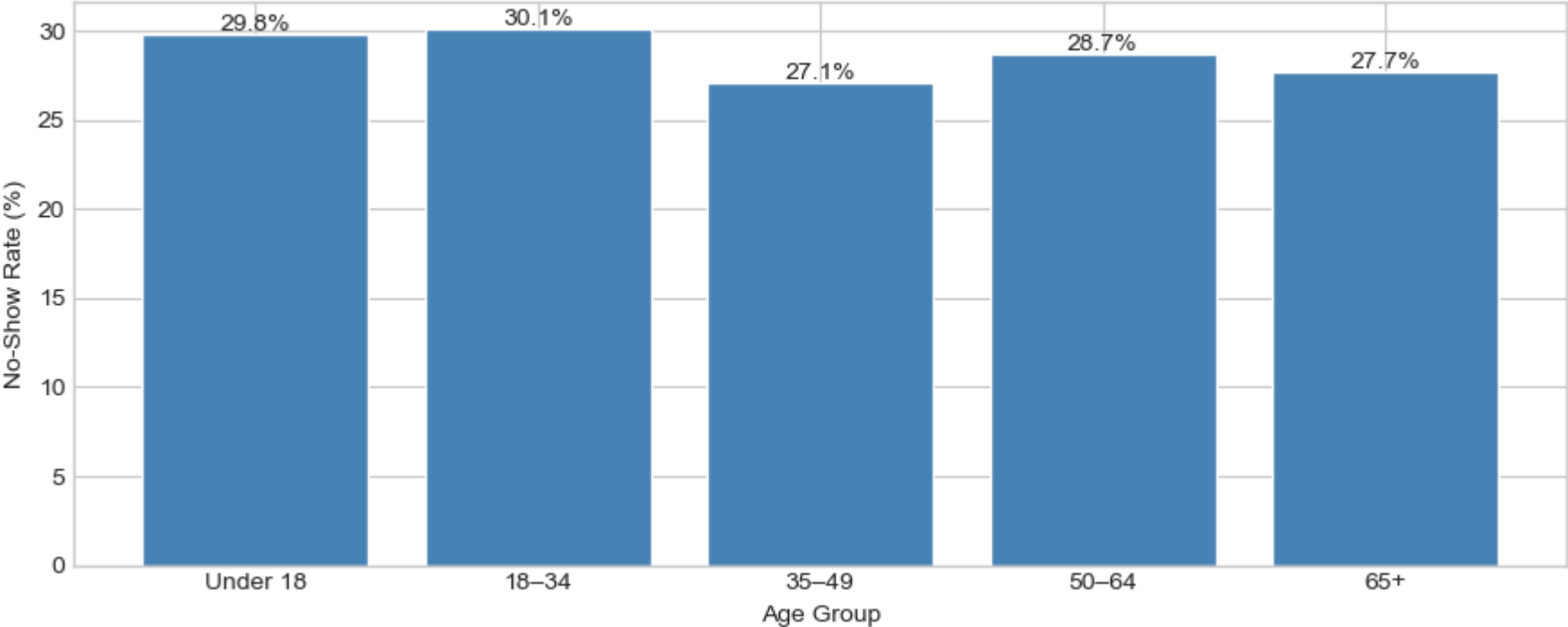
4. No-Show Rate by Age Group

```
In [5]: result = sql("""
SELECT
    CASE
        WHEN Age < 18 THEN 'Under 18'
        WHEN Age < 35 THEN '18-34'
        WHEN Age < 50 THEN '35-49'
        WHEN Age < 65 THEN '50-64'
        ELSE '65+'
    END AS age_group,
    COUNT(*) AS total,
    SUM(No_show) AS no_shows,
    ROUND(AVG(No_show) * 100, 1) AS no_show_rate_pct
FROM appointments
GROUP BY age_group
ORDER BY no_show_rate_pct DESC
""")
print(result.to_string(index=False))

plt.figure(figsize=(9, 4))
order = ['Under 18', '18-34', '35-49', '50-64', '65+']
result_sorted = result.set_index('age_group').reindex(order).reset_index()
plt.bar(result_sorted['age_group'], result_sorted['no_show_rate_pct'], color='steelblue', edgecolor='white')
plt.title('No-Show Rate by Age Group', fontsize=13)
plt.xlabel('Age Group')
plt.ylabel('No-Show Rate (%)')
for i, row in result_sorted.iterrows():
    plt.text(i, row['no_show_rate_pct'] + 0.2, f"{row['no_show_rate_pct']}%", ha='center', fontsize=10)
plt.tight_layout()
plt.show()
```

age_group	total	no_shows	no_show_rate_pct
18-34	1254	378	30.1
Under 18	2246	669	29.8
50-64	457	131	28.7
65+	842	233	27.7
35-49	701	190	27.1

No-Show Rate by Age Group



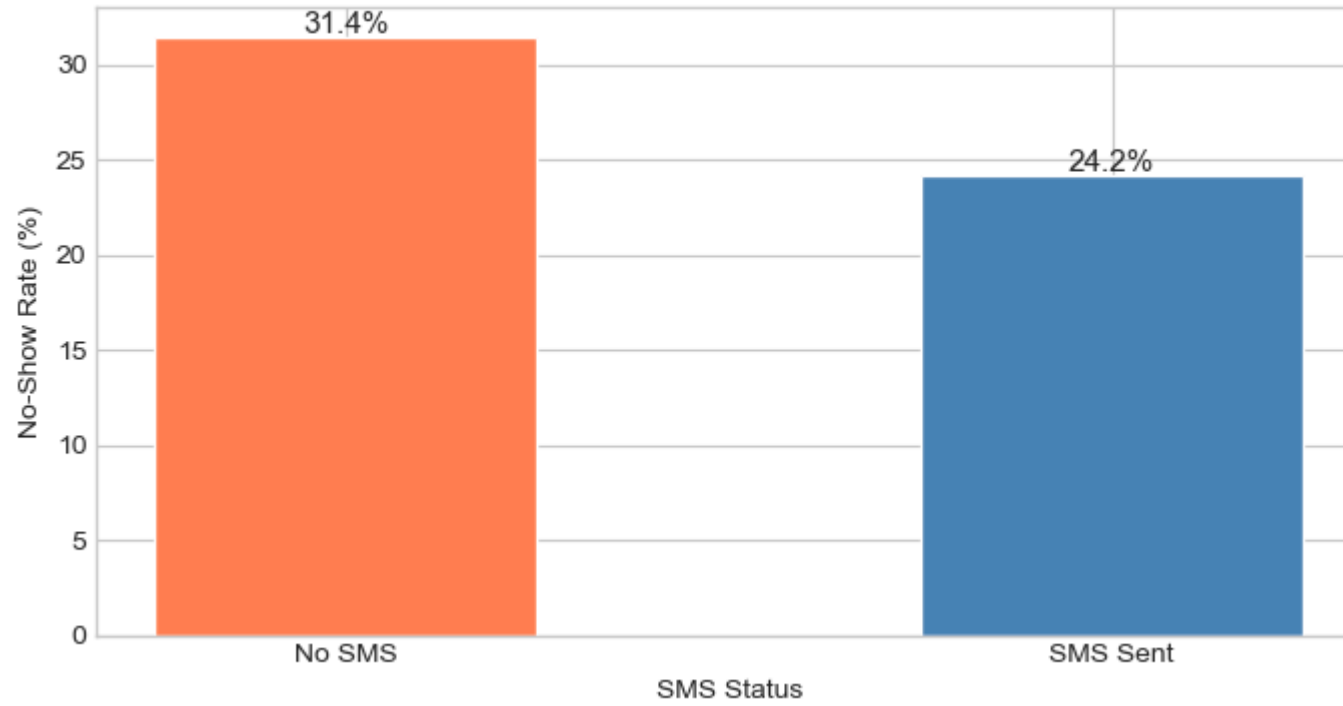
5. Impact of SMS Reminders

```
In [6]: result = sql("""
SELECT
    CASE WHEN SMS_received = 1 THEN 'SMS Sent' ELSE 'No SMS' END AS sms_status,
    COUNT(*) AS total,
    SUM(No_show) AS no_shows,
    ROUND(AVG(No_show) * 100, 1) AS no_show_rate_pct
FROM appointments
GROUP BY SMS_received
ORDER BY SMS_received
""")
print(result.to_string(index=False))

plt.figure(figsize=(7, 4))
colors = ['coral', 'steelblue']
plt.bar(result['sms_status'], result['no_show_rate_pct'], color=colors, edgecolor='white', width=0.5)
plt.title('No-Show Rate: SMS Reminder vs No Reminder', fontsize=13)
plt.xlabel('SMS Status')
plt.ylabel('No-Show Rate (%)')
for i, row in result.iterrows():
    plt.text(i, row['no_show_rate_pct'] + 0.2, f"{row['no_show_rate_pct']}%", ha='center', fontsize=11)
plt.tight_layout()
plt.show()
```

sms_status	total	no_shows	no_show_rate_pct
No SMS	3766	1181	31.4
SMS Sent	1734	420	24.2

No-Show Rate: SMS Reminder vs No Reminder



6. Impact of Waiting Time on No-Shows

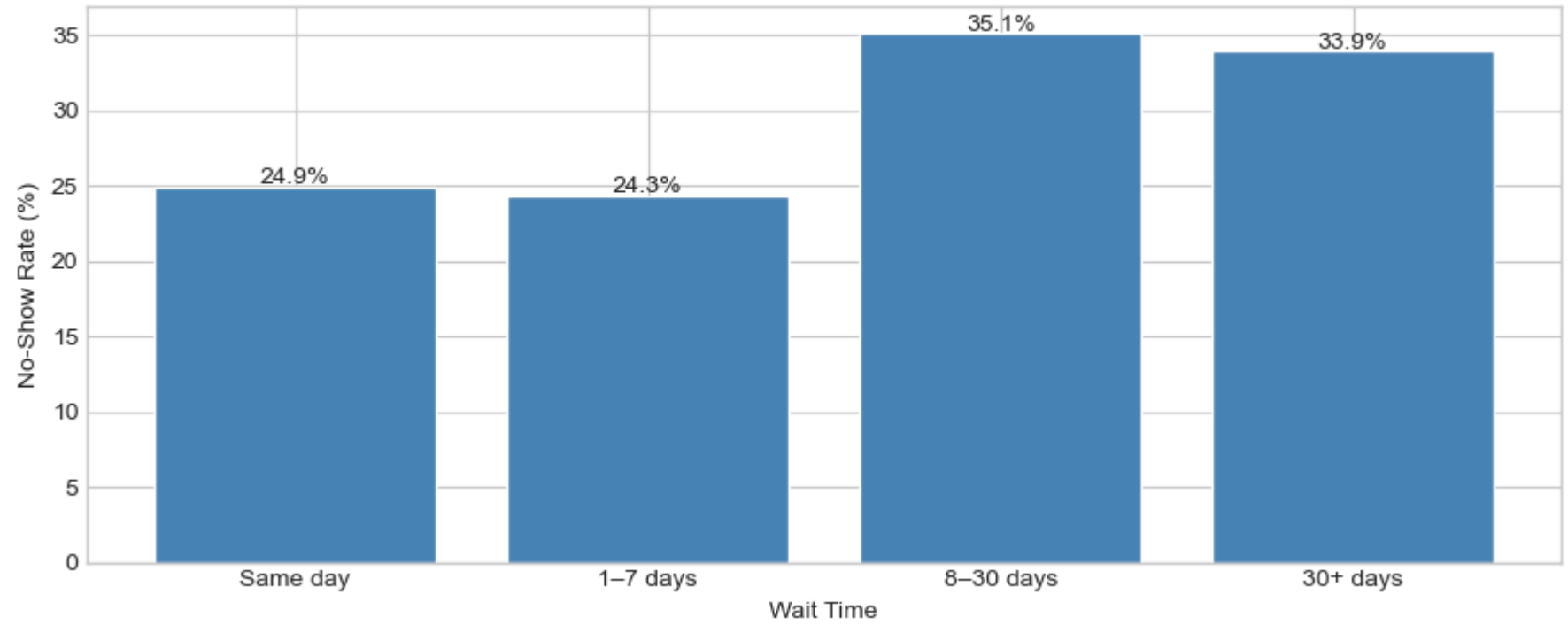
```
In [7]: result = sql("""
SELECT
    CASE
        WHEN Wait_Days = 0 THEN 'Same day'
        WHEN Wait_Days <= 7 THEN '1-7 days'
        WHEN Wait_Days <= 30 THEN '8-30 days'
        ELSE '30+ days'
    END AS wait_group,
    COUNT(*) AS total,
    SUM(No_show) AS no_shows,
    ROUND(AVG(No_show) * 100, 1) AS no_show_rate_pct
FROM appointments
GROUP BY wait_group
ORDER BY no_show_rate_pct
""")
print(result.to_string(index=False))

order = ['Same day', '1-7 days', '8-30 days', '30+ days']
result_sorted = result.set_index('wait_group').reindex(order).reset_index()

plt.figure(figsize=(9, 4))
plt.bar(result_sorted['wait_group'], result_sorted['no_show_rate_pct'], color='steelblue', edgecolor='white')
plt.title('No-Show Rate by Waiting Time', fontsize=13)
plt.xlabel('Wait Time')
plt.ylabel('No-Show Rate (%)')
for i, row in result_sorted.iterrows():
    plt.text(i, row['no_show_rate_pct'] + 0.2, f"{row['no_show_rate_pct']}%", ha='center', fontsize=10)
plt.tight_layout()
plt.show()
```

wait_group	total	no_shows	no_show_rate_pct
1-7 days	2515	612	24.3
Same day	543	135	24.9
30+ days	251	85	33.9
8-30 days	2191	769	35.1

No-Show Rate by Waiting Time



7. No-Shows by Health Conditions

```

In [8]: result = sql("""
SELECT
    'Hypertension' AS condition,
    ROUND(AVG(CASE WHEN Hypertension=1 THEN No_show END)*100, 1) AS with_condition,
    ROUND(AVG(CASE WHEN Hypertension=0 THEN No_show END)*100, 1) AS without_condition
FROM appointments

UNION ALL

SELECT
    'Diabetes',
    ROUND(AVG(CASE WHEN Diabetes=1 THEN No_show END)*100, 1),
    ROUND(AVG(CASE WHEN Diabetes=0 THEN No_show END)*100, 1)
FROM appointments

UNION ALL

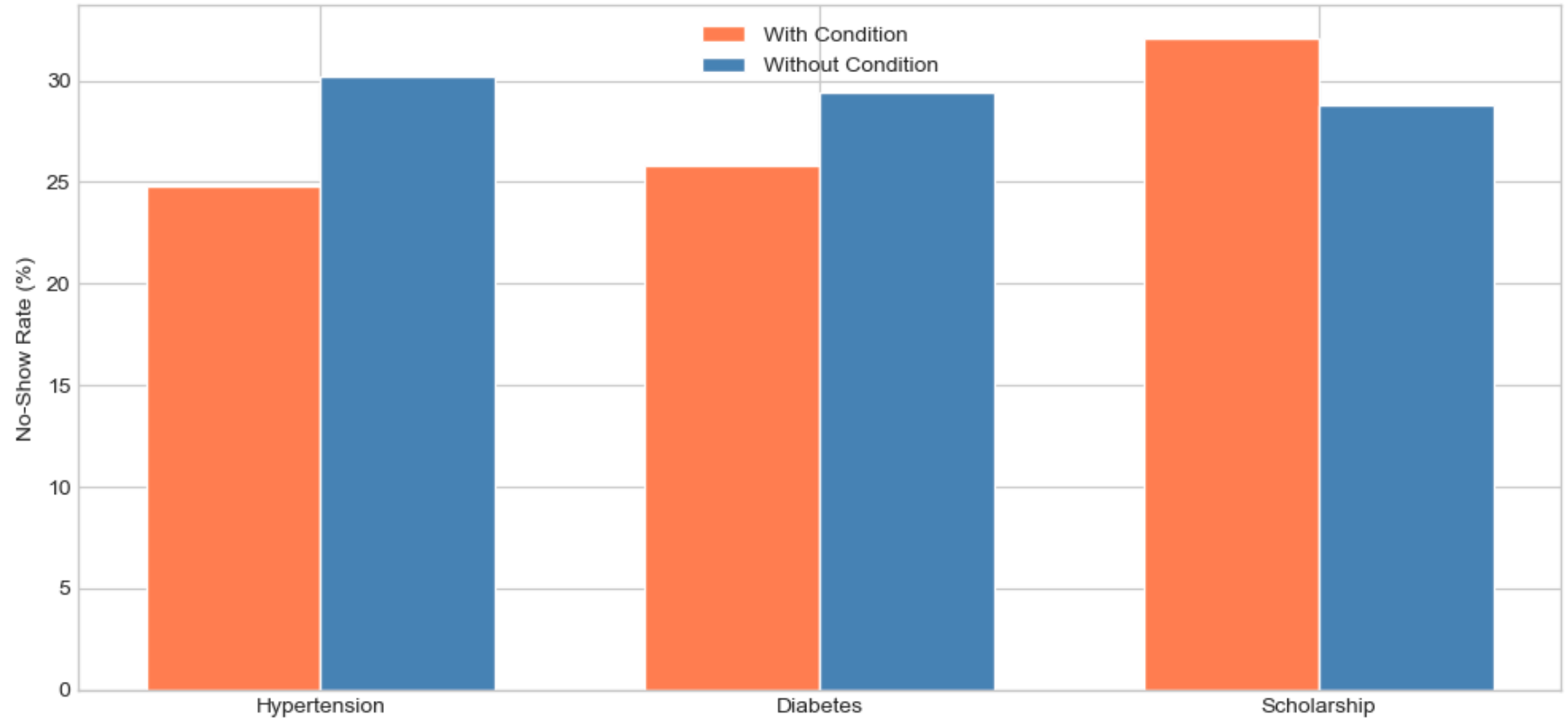
SELECT
    'Scholarship',
    ROUND(AVG(CASE WHEN Scholarship=1 THEN No_show END)*100, 1),
    ROUND(AVG(CASE WHEN Scholarship=0 THEN No_show END)*100, 1)
FROM appointments
""")
print(result.to_string(index=False))

x = np.arange(len(result))
width = 0.35
plt.figure(figsize=(10, 5))
plt.bar(x - width/2, result['with_condition'], width, label='With Condition', color='coral', edgecolor='white')
plt.bar(x + width/2, result['without_condition'], width, label='Without Condition', color='steelblue', edgecolor='white')
plt.xticks(x, result['condition'])
plt.title('No-Show Rate: With vs Without Health Condition / Scholarship', fontsize=13)
plt.ylabel('No-Show Rate (%)')
plt.legend()
plt.tight_layout()
plt.show()

```

condition	with_condition	without_condition
Hypertension	24.8	30.2
Diabetes	25.8	29.4
Scholarship	32.1	28.8

No-Show Rate: With vs Without Health Condition / Scholarship



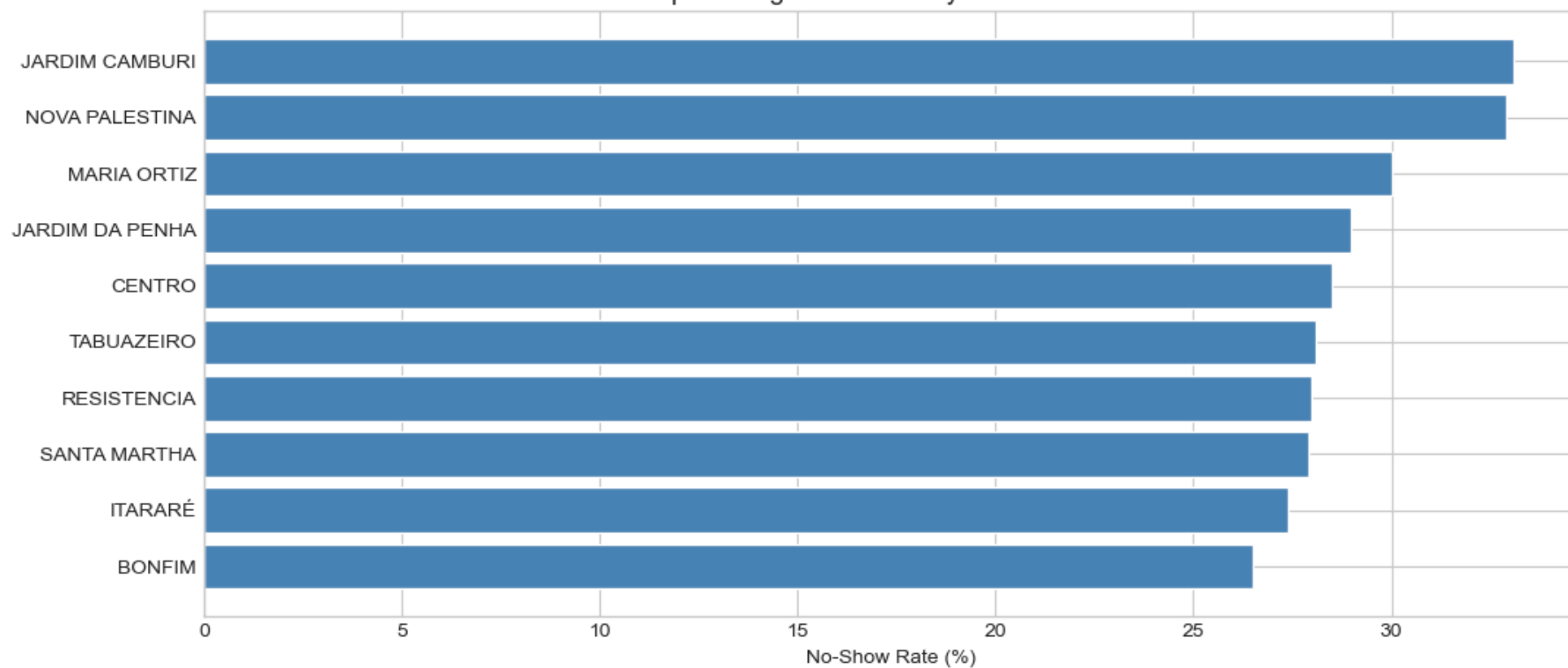
8. No-Show Rate by Neighbourhood

```
In [9]: result = sql("""
SELECT
    Neighbourhood,
    COUNT(*)                AS total,
    SUM(No_show)            AS no_shows,
    ROUND(AVG(No_show) * 100, 1) AS no_show_rate_pct
FROM appointments
GROUP BY Neighbourhood
HAVING COUNT(*) > 50
ORDER BY no_show_rate_pct DESC
LIMIT 10
""")
print(result.to_string(index=False))

plt.figure(figsize=(11, 5))
plt.barh(result['Neighbourhood'], result['no_show_rate_pct'], color='steelblue', edgecolor='white')
plt.gca().invert_yaxis()
plt.title('Top 10 Neighbourhoods by No-Show Rate', fontsize=13)
plt.xlabel('No-Show Rate (%)')
plt.tight_layout()
plt.show()
```

Neighbourhood	total	no_shows	no_show_rate_pct
JARDIM CAMBURI	537	178	33.1
NOVA PALESTINA	532	175	32.9
MARIA ORTIZ	540	162	30.0
JARDIM DA PENHA	542	157	29.0
CENTRO	585	167	28.5
TABUAZEIRO	530	149	28.1
RESISTENCIA	539	151	28.0
SANTA MARTHA	578	161	27.9
ITARARÉ	566	155	27.4
BONFIM	551	146	26.5

Top 10 Neighbourhoods by No-Show Rate



9. Advanced SQL — Window Functions & Ranking

```
In [10]: # Rank neighbourhoods by no-show rate using window functions
result = sql("""
WITH neighbourhood_stats AS (
    SELECT
        Neighbourhood,
        COUNT(*)                AS total,
        SUM(No_show)             AS no_shows,
        ROUND(AVG(No_show)*100, 1) AS no_show_rate_pct
    FROM appointments
    GROUP BY Neighbourhood
    HAVING COUNT(*) > 50
)
SELECT
    Neighbourhood,
    total,
    no_shows,
    no_show_rate_pct,
    RANK() OVER (ORDER BY no_show_rate_pct DESC) AS rank_worst
FROM neighbourhood_stats
ORDER BY rank_worst
""")
print(result.to_string(index=False))
```

Neighbourhood	total	no_shows	no_show_rate_pct	rank_worst
JARDIM CAMBURI	537	178	33.1	1
NOVA PALESTINA	532	175	32.9	2
MARIA ORTIZ	540	162	30.0	3
JARDIM DA PENHA	542	157	29.0	4
CENTRO	585	167	28.5	5
TABUAZEIRO	530	149	28.1	6
RESISTENCIA	539	151	28.0	7
SANTA MARTHA	578	161	27.9	8
ITARARÉ	566	155	27.4	9
BONFIM	551	146	26.5	10

```
In [11]: # Patients with multiple no-shows (high-risk patients)
result = sql("""
SELECT
    PatientId,
    COUNT(*)                AS total_appointments,
    SUM(No_show)            AS total_no_shows,
    ROUND(AVG(No_show)*100, 1) AS personal_no_show_rate_pct
FROM appointments
GROUP BY PatientId
HAVING total_appointments >= 3
      AND total_no_shows >= 2
ORDER BY total_no_shows DESC
LIMIT 10
""")
print("High-risk repeat no-show patients:")
print(result.to_string(index=False))
```

High-risk repeat no-show patients:

Empty DataFrame

Columns: [PatientId, total_appointments, total_no_shows, personal_no_show_rate_pct]

Index: []

10. Summary & Conclusions

Key SQL Findings

Waiting Time is the Strongest Predictor:

- Patients waiting 30+ days have significantly higher no-show rates than same-day appointments.
- Reducing wait times is the most impactful lever for improving attendance.

SMS Reminders Work:

- Patients who did **not** receive an SMS reminder had a notably higher no-show rate.
- This is an actionable, low-cost intervention clinics can implement immediately.

Younger Patients (18–34) are Most Likely to No-Show:

- Older patients (65+) are the most reliable attenders, likely due to the seriousness of their conditions.

- Targeted outreach for younger demographics could improve outcomes.

Chronic Conditions Reduce No-Shows:

- Patients with hypertension or diabetes are less likely to miss appointments — they depend on consistent care.
- Scholarship recipients (lower income) show slightly higher no-show rates, suggesting transport or cost barriers.

SQL Techniques Demonstrated

- GROUP BY with aggregates (COUNT , SUM , AVG)
- CASE WHEN for conditional grouping and pivoting
- HAVING clause for post-aggregation filtering
- UNION ALL for combining multiple query results
- **Window functions** (RANK() OVER) for ranking
- **CTEs** (WITH clause) for readable, modular queries